

Projektdokumentation

pxsng The AI Training Manager

Ein Werkzeug zum Verwalten von Schulungen mit KI Unterstützung

Team Tim, Fynn und Marco

Stand April 2026

Projekt auf GitHub unter github.com/TimLMueller/pxsng

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Einleitung	5
1.1 Projektbeschreibung	5
1.2 Ausgangssituation	5
1.3 Zielsetzung	5
1.4 Abgrenzung	5
2. Anforderungsanalyse	7
2.1 Fachliche Anforderungen	7
2.2 Technische Anforderungen	7
2.3 Geschäftsprozesse	8
2.4 Risiken	8
3. Entwurf	9
3.1 Systemarchitektur	9
3.2 Datenbankmodell	9
3.3 API Design	10
3.4 Use Case Diagramm	10
3.5 Ablaufdiagramm	11
3.6 Designentscheidungen	12
4. Implementierung	13
4.1 Projektstruktur	13
4.2 Beschreibung des Codes	13
4.2.1 Frontend	13
4.2.2 Backend	13
4.2.3 Datenbank	14
4.2.4 Deployment	14
4.3 API Endpunkte	14
4.4 Saubere Codeprinzipien	14
4.5 Unfertige Module	15
5. Test und Qualitätssicherung	17
5.1 Teststrategie	17
5.2 Durchgeführte Tests	17
5.3 Testergebnisse	17
5.4 Probleme beim Testen	17
6. Deployment und Installation	19
6.1 Installation	19

6.2 Konfiguration	19
6.3 Nutzung	19
7. Entwicklungsprozess und Reflexion.....	20
7.1 Vorgehensweise.....	20
7.2 Aufgetretene Probleme	20
7.3 Wie wir das ORDS Problem gelöst haben	21
7.4 Reflexion	21
8. Entscheidungen und Begründungen.....	22
8.1 Technologische Entscheidungen	22
8.2 Architekturentscheidungen	22
8.3 Fachliche Entscheidungen.....	22
9. Übergabe an die nächste Klasse	23
9.1 Offene Punkte	23
9.2 Zusätzliche Erweiterungen	23
9.3 Tipps für die nächste Klasse	24
10. Fazit.....	25
10.1 Zusammenfassung	25
10.2 Was wir persönlich mitnehmen.....	25
Anhang A. Diagramme und Entwürfe.....	26
A.1 ER Diagramm	26
A.2 Aktivitätsdiagramm	27
Anhang B. Anleitung für Anwender und Entwickler	30
B.1 Anleitung für Anwender	30
B.1.1 Frontend öffnen.....	30
B.1.2 Was schon geht	30
B.1.3 Was noch nicht geht	30
B.2 Anleitung für Entwickler	30
B.2.1 Voraussetzungen	30
B.2.2 Repository klonen	31
B.2.3 Datenbank starten und Stack hochfahren.....	31
B.2.4 Backend mit echter Datenbank verwenden	31
B.2.5 Frontend lokal entwickeln	32
B.2.6 Tests und Linting.....	32
B.2.7 Datenbank zurücksetzen	32
B.2.8 Wo am besten anfangen	32
Anhang C. Auszüge aus dem Code.....	33
C.1 Aufruf des PL SQL Pakets im Backend	33

C.2 Service im Frontend	33
C.3 Auszug aus dem Datenbank Setup	33

1. Einleitung

1.1 Projektbeschreibung

In diesem Schulprojekt haben wir angefangen, ein Werkzeug zu bauen, mit dem Firmen ihre Schulungen verwalten können. Es heißt pxsng The AI Training Manager. Das Werkzeug soll Kurse, Teilnehmer und Firmen zentral verwalten und auch die Angebote, Rechnungen und Zertifikate übernehmen, die dabei anfallen. Außerdem soll eine lokale KI dabei helfen, immer wiederkehrende Texte automatisch zu schreiben.

Diese Dokumentation beschreibt, wie weit wir am Ende des Schuljahres gekommen sind. Sie ist vor allem für die nächste Klasse gedacht, die unser Projekt weiterführen soll. Deshalb haben wir besonders darauf geachtet, ehrlich aufzuschreiben, was schon funktioniert, was nur halb fertig ist und was wir gar nicht mehr geschafft haben.

1.2 Ausgangssituation

In vielen Firmen werden Schulungen, Teilnehmerlisten und Rechnungen heute noch mit Excel, Word und E Mail verwaltet. Das ist viel Handarbeit und es schleichen sich leicht Fehler ein. Gleichzeitig kann man heute mit einer KI viele Standardtexte automatisch erzeugen lassen, was bei vielen Firmen aber noch nicht ankommt. Genau hier wollten wir mit unserem Projekt ansetzen.

1.3 Zielsetzung

Unser Ziel war eine Anwendung in mehreren Teilen. Ein Frontend für die Teilnehmer, ein Bereich für Admins, ein Backend in Python, ein PDF Dienst und ein KI Modul sowie eine Datenbank in Oracle, die einen Teil der Logik selbst übernimmt. Alles sollte mit Docker laufen, damit man es überall gleich starten kann. Welche Funktionen genau geplant waren, steht im nächsten Kapitel.

1.4 Abgrenzung

Schon am Anfang war klar, dass wir nicht alles bauen können. Den Versand und Empfang von E Mails haben wir bewusst weggelassen, ebenso eine echte

Bezahlung über einen Anbieter wie PayPal und einen vollwertigen Kalender mit Terminen. Diese Sachen wären für ein Schulprojekt zu groß gewesen.

2. Anforderungsanalyse

2.1 Fachliche Anforderungen

Wir haben unsere Anforderungen im Exposé festgehalten und sie in Mussziele und Kannziele unterteilt. Die Mussziele sind die Funktionen, die das Projekt am Ende auf jeden Fall haben sollte. Die Kannziele sind die zusätzlichen Funktionen, die wir nur einbauen wollten, wenn am Ende noch Zeit übrig ist. Die folgende Tabelle zeigt, wie weit wir mit den einzelnen Punkten gekommen sind.

Typ	Anforderung	Stand April 2026
Muss	Kurse, Firmen und Teilnehmer verwalten (anlegen, ändern, löschen)	Teilweise. Die Datenbank ist fertig, das Backend kann nur lesen, das Adminfrontend fehlt komplett
Muss	Angebot, Rechnung und Zertifikat als PDF erstellen	Offen
Muss	Texte für Angebote und Rechnungen mit KI erzeugen	Offen
Muss	Dashboard für Admins mit Umsatz und offenen Rechnungen	Offen
Muss	Teilnehmerportal mit Kursübersicht und gebuchten Kursen	Teilweise. Eine Startseite mit Kurskarten ist da
Kann	Kursempfehlungen durch KI	Offen
Kann	Zusammenfassung von Feedback durch KI	Offen
Kann	Erweiterte Statistiken im Adminbereich	Offen

2.2 Technische Anforderungen

Im Exposé hatten wir uns auf einen ganz bestimmten Technologiestack festgelegt. Das Frontend sollte mit React gebaut werden, das Backend mit Python und Flask, die Datenbank ist Oracle und zwischen Backend und Datenbank sollte [ORDS](#) sitzen. ORDS ist eine Schicht von Oracle, die Datenbankfunktionen direkt als REST

verfügbar macht. Alles zusammen sollte mit Docker laufen, damit es auf jedem Rechner gleich startet. Im Laufe des Projekts mussten wir bei einem Punkt umdenken, was wir in [Kapitel 8](#) ausführlich beschreiben.

2.3 Geschäftsprozesse

Aus den fachlichen Anforderungen haben wir vier Geschäftsprozesse abgeleitet, die wir im System abbilden wollten. Zuerst legt ein Admin die Kurse an und pflegt sie. Dann bucht ein Teilnehmer oder eine Firma einen Kurs. Danach werden ein Angebot und eine Rechnung erstellt. Am Ende, wenn der Kurs abgeschlossen ist, bekommt der Teilnehmer ein Zertifikat. Das passende Aktivitätsdiagramm zu diesem Ablauf liegt in [Anhang A](#).

2.4 Risiken

Schon am Anfang war uns klar, dass es ein paar Stellen gibt, an denen es schwierig werden kann. Erstens ist Oracle keine typische Datenbank für ein Schulprojekt. Das Image ist groß, das Setup ist umständlich und ORDS macht oft Probleme. Zweitens haben wir uns insgesamt sehr viel vorgenommen, weil zum klassischen CRUD System auch noch ein PDF Dienst und eine KI dazukommen sollten. Drittens hatte keiner von uns vorher mit Oracle oder ORDS gearbeitet. Wie sich später gezeigt hat, sind genau diese Risiken auch eingetreten. Mehr dazu in [Kapitel 7](#).

3. Entwurf

3.1 Systemarchitektur

Unser System ist klassisch in drei Schichten aufgebaut. Im Browser läuft das Frontend, das mit React geschrieben ist. Das Frontend redet per HTTP mit dem Backend, das in Python mit Flask läuft. Das Backend wiederum redet mit der Oracle Datenbank, in der zusätzlich Geschäftslogik in Form von PL SQL Paketen liegt. Frontend, Backend und Datenbank laufen alle in einem eigenen Container und werden zusammen über docker compose gestartet.

Im ursprünglichen Plan sollte zwischen Backend und Datenbank noch ORDS dazwischen sitzen. ORDS war also von Anfang an Teil unseres geplanten Stacks. Wir haben den ORDS Eintrag im docker compose deshalb stehen gelassen, aber auskommentiert. Warum wir ORDS im Laufe der Entwicklung haben sein lassen, steht in [Kapitel 7](#) und in [Kapitel 8](#).

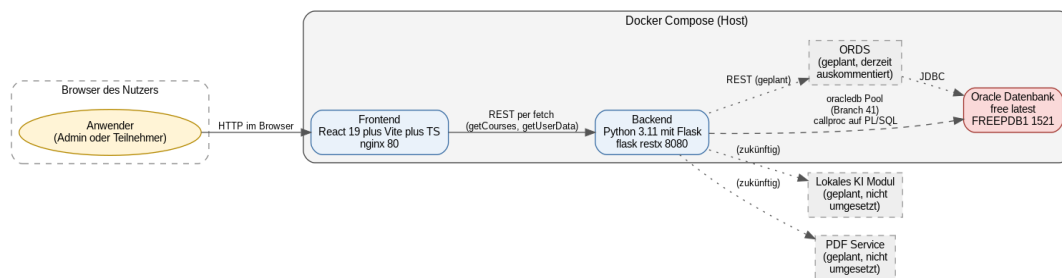


Abbildung 1. Aufbau des Systems mit den geplanten und tatsächlich gebauten Teilen

3.2 Datenbankmodell

Unsere Datenbank hat 16 Tabellen. Sie deckt alle Bereiche ab, die wir brauchen. Stammdaten wie Land, Postleitzahl und Adresse, dann Personen wie User, Teilnehmer, Firma und Referral, dann alles rund um Kurse und Buchungen, dann die kaufmännischen Daten wie Rechnung und Zertifikat, und am Ende noch den Warenkorb. Insgesamt gibt es 22 Fremdschlüssel. Drei davon mussten wir als deferrable anlegen, weil zwischen User, Firma und Referral ein Kreis besteht und die Datenbank sich sonst über die Reihenfolge beim Einfügen beschwert. Das komplette ER Diagramm ist in [Anhang A](#) zu sehen.

Ein Teil der Logik liegt direkt in der Datenbank, und zwar in zwei PL SQL Paketen. Das Paket USER_API_PKG kümmert sich um die User und kann sie auslesen,

anlegen und löschen. Das Paket COURSE_API_PKG macht das Gleiche für Kurse. Zusätzlich haben wir sieben Trigger, die das Feld UPDATED_AT automatisch setzen, wenn ein Datensatz geändert wird.

3.3 API Design

Unsere REST Schnittstelle ist mit flask restx gebaut. Das hat den Vorteil, dass wir automatisch eine Swagger Oberfläche bekommen, in der man sich alle Endpunkte ansehen kann. Im Moment sind die folgenden Endpunkte definiert.

Methode	Pfad	Funktion	Stand
GET	/courses	Alle Kurse holen	Fertig. Aktuell mit Mockdaten, die echte Datenbankbindung ist im Code vorbereitet
GET	/courses/:id	Einen Kurs anhand der ID holen	Fertig
GET	/users	Alle User holen	Fertig (mit Mockdaten)
GET	/users/:id	Einen User anhand der ID holen	Fertig
PUT	/users	Einen neuen User anlegen	Skelett da, ohne Prüfung der Daten
POST PUT DELETE	Buchungen, Angebote, Rechnungen, Zertifikate	Geplant, noch nicht gebaut	Offen

3.4 Use Case Diagramm

Das Use Case Diagramm auf der nächsten Seite zeigt, welche Personen mit welchen Funktionen unseres Systems arbeiten. Funktionen, die wir schon halbwegs gebaut haben, sind mit einer durchgezogenen Linie eingezeichnet. Funktionen, die noch fehlen, sind gestrichelt. So sieht man auf einen Blick, wie viel noch offen ist.

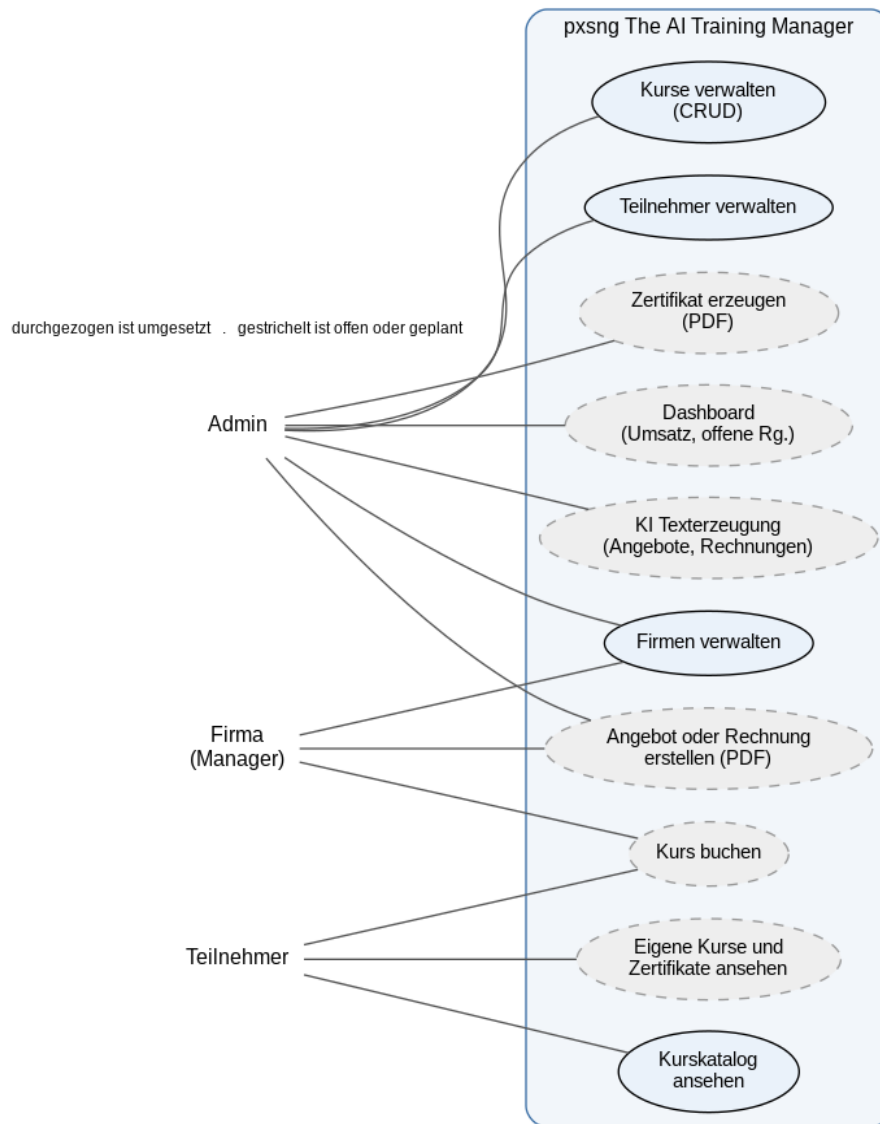


Abbildung 2. Use Case Diagramm mit fertigen und offenen Funktionen

3.5 Ablaufdiagramm

Der wichtigste Ablauf in unserem System ist das Abrufen der Kursliste, weil dabei alle Schichten zusammenarbeiten. Der Browser fragt das Frontend, das Frontend fragt das Backend, das Backend fragt die Datenbank. Die folgende Skizze zeigt sowohl den geplanten Weg über ORDS (gestrichelt) als auch den Weg, den wir am Ende tatsächlich gegangen sind (durchgezogen). Ein detailliertes Aktivitätsdiagramm zum Buchungsprozess liegt zusätzlich in [Anhang A](#).

Ablauf Kurs abrufen (Sollzustand mit ORDS, Istzustand ohne ORDS)

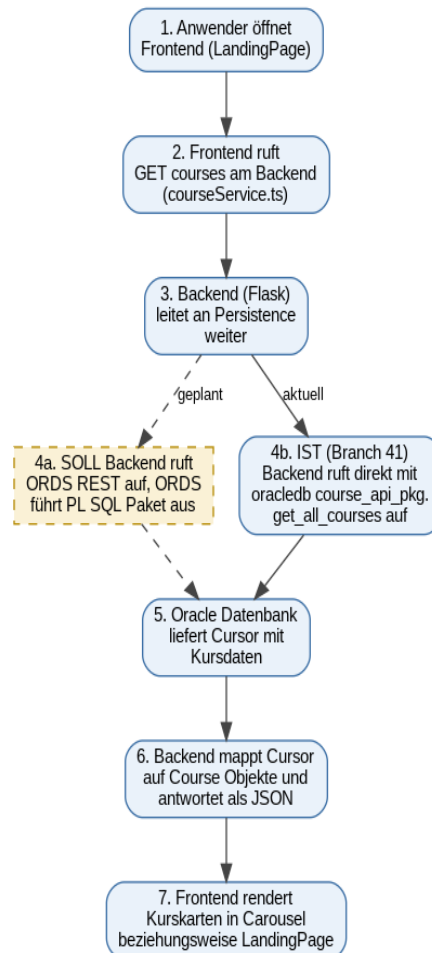


Abbildung 3. Ablauf beim Abrufen der Kurse

3.6 Designentscheidungen

Die wichtigste Entscheidung im Entwurf war, ORDS im Laufe der Entwicklung sein zu lassen. Stattdessen redet das Backend direkt mit der Datenbank, und zwar mit dem Pythontreiber oracledb. Die Aufrufe an die PL SQL Pakete bleiben dabei genau gleich. Die ausführliche Begründung steht in [Kapitel 8](#).

4. Implementierung

4.1 Projektstruktur

So ist unser Projekt im Repository aufgebaut.

pxsng/	
frontend/	React 19 + Vite + TS + MUI + Tailwind
src/	
app/	App.tsx, Compose.tsx
components/	TopBar, Course, Carousel ...
pages/	LandingPage
providers/	UserProvider, SettingsProvider ...
services/	courseService.ts, userService.ts
backend/	Flask + flask-restx
src/	
Controllers/	CourseController, UserController
Persistence/	DatabaseConnection, CoursesCalls, UsersCalls
Types/	Course, User
main.py	Einstieg der App, registriert Namespaces
model/	
oracle-data/	Volume + setup_db_core.sql + setup.sh
ords-data/	Konfig (zur Zeit nicht in Benutzung)
deployment/	docker-compose.yml + .env.example
docs/	expose.pdf, outline.md, Diagramme

4.2 Beschreibung des Codes

4.2.1 Frontend

Unser Frontend ist eine ganz aktuelle React 19 Anwendung. Als Buildwerkzeug benutzen wir Vite, das viel schneller startet als Webpack. Für die Optik haben wir Tailwind und Material UI verwendet. In der Datei App.tsx werden die TopBar und die LandingPage mit React.lazy nachgeladen, damit die Seite schneller fertig ist. Die LandingPage zeigt ein Karussell mit Kurskarten. Die Daten dafür holt sie über die Datei courseService.ts, die per fetch GET /courses am Backend aufruft. Der SettingsProvider kümmert sich darum, ob die Seite hell oder dunkel angezeigt wird. Der UserProvider ist nur ein leeres Gerüst, weil wir das Login noch nicht eingebaut haben.

4.2.2 Backend

Das Backend haben wir mit Flask und flask restx gebaut. In der Datei main.py erstellen wir die Flask App, hängen flask restx dran und registrieren drei Namespaces, einen für Kurse und, einen für User und einen für den CurrentUser was aktuell als unser eingeloggter User dient. Die Controller schreiben mit dem

Dekorator `@api.route` die Routen sehr kurz hin. Für die echte Datenbankanbindung haben wir zusätzlich eine Persistenzschicht geschrieben. Die Datei `DatabaseConnection.py` baut mit `oracledb.create_pool` einen Connectionpool auf, `CoursesCalls.py` und `UsersCalls` ruft mit `cursor.callproc` die PL SQL Pakete direkt auf. Die nächste Klasse muss diese Persistenzschicht nur an den restlichen Routen ergänzen, dann sind die Mockdaten überall ersetzt.

4.2.3 Datenbank

Die Datenbank bauen wir komplett über die Datei `setup_db_core.sql` im Ordner `model` auf. Das Skript legt einen Schemauser an, erstellt 16 Tabellen, 27 Indizes, die beiden PL SQL Pakete und sieben Trigger für `UPDATED_AT`. Aufgerufen wird das Skript automatisch von `setup.sh`, das beim Start des Containers ausgeführt wird. Eine genaue Beschreibung des Skripts liegt im selben Ordner unter dem Namen `USAGE_GUIDE.md`.

4.2.4 Deployment

Im Ordner `deployment` liegt unsere Composedatei und eine Beispielkonfiguration unter dem Namen `.env.example`. Compose startet drei Container, nämlich das Frontend auf Port 80, das Backend auf Port 8080 und die Oracle Datenbank im internen Netz. Die Volumes der Datenbank zeigen auf den Ordner `model`, sodass das Setupskript beim ersten Start automatisch ausgeführt wird. Der ORDS Eintrag steht zwar noch in der Datei, ist aber auskommentiert.

4.3 API Endpunkte

Welche Endpunkte unser Backend hat, steht in der Tabelle in [Kapitel 3.3](#). Da flask restx automatisch eine OpenAPI Beschreibung erzeugt, gibt es immer eine aktuelle Swagger Oberfläche, sobald das Backend läuft. Man kann sie unter <http://localhost:8080> öffnen, wenn der Port nach außen freigegeben ist.

4.4 Saubere Codeprinzipien

Wir haben versucht, unseren Code in einzelne Schichten zu trennen. Im Backend gibt es Controller, Persistence und Types als getrennte Ordner, sodass die Datenbankaufrufe nicht direkt in den Routen stehen. Im Frontend sind die Komponenten, Seiten und Backendaufrufe ebenfalls getrennt. Im Frontend läuft beim

Commit automatisch eslint und prettier dank Husky, sodass der Code immer einheitlich formatiert ist. Im Backend gibt es noch kein einheitliches Linting, das sollte die nächste Klasse als Erstes einbauen.

4.5 Unfertige Module

Damit die nächste Klasse weiß, wo sie ansetzen muss, listen wir hier alle Module auf, die noch nicht oder nur halb fertig sind.

Modul	Stand	Nächste Schritte
ORDS	Container und Composeeintrag liegen vor, sind aber auskommentiert	Entweder das Setup nochmal versuchen und stabil zum Laufen bringen oder ORDS endgültig entfernen
PDF Dienst	Noch nicht angefangen	Eigenes Modul im Backend bauen, zum Beispiel mit reportlab oder WeasyPrint, das Angebote, Rechnungen und Zertifikate erzeugt
KI Modul	Noch nicht angefangen	Lokales Modell anbinden, zum Beispiel über Ollama, dazu einen Endpunkt /ai/generate mit fertigen Prompts bauen
Adminfrontend	Noch nicht vorhanden	Entweder ein eigenes Reactprojekt für die Admins oder geschützte Routen unter /admin im bestehenden Frontend, dazu Formulare für Kurs, Firma und Teilnehmer
Login und Auth	UserProvider ist nur ein Gerüst	Loginendpunkt im Backend, JWT zur Authentifizierung und Routenschutz im Frontend
Tests	Vitest ist eingerichtet, es gibt aber keine Tests	Tests für Services und Komponenten im Frontend schreiben, pytest im Backend einführen

5. Test und Qualitätssicherung

5.1 Teststrategie

Eine richtige Teststrategie mit automatischen Tests haben wir im Schuljahr leider nicht mehr aufgebaut. Wir haben unseren Code überwiegend manuell als sogenannte Smoketests geprüft. Das heißt, wir haben den Stack mit docker compose gestartet und dann von Hand geschaut, ob das Frontend lädt, das Backend antwortet und die Datenbank die richtigen Daten hat. Im Frontend ist mit vitest schon ein Testframework eingerichtet, es gibt aber noch keine Tests. Im Backend fehlt das Testsetup komplett.

5.2 Durchgeführte Tests

Bei der Datenbank haben wir manuell geprüft, ob unser Setupskript auf einer frischen Datenbank komplett durchläuft, ob die beiden PL SQL Pakete fehlerfrei kompiliert werden und ob die Beispielprozedur create_user_record einen User samt Warenkorb anlegt. Die passende Testsequenz steht im USAGE_GUIDE.md neben dem Skript. Beim Backend haben wir die Datenbankanbindung lokal ausprobiert und über den Endpunkt GET /courses echte Daten aus dem PL SQL Paket geholt. Gleichzeitig haben wir hierbei die kritischen Treiber-Deadlocks bei den ID-Abrufen analysiert und erfolgreich gelöst in Kapitel [5.4](#).

5.3 Testergebnisse

Das Setup der Datenbank läuft stabil. Wenn man den Stack neu hochfährt, geht alles durch. Das Backend mit den Mockdaten läuft ebenfalls stabil und liefert immer die gleiche Antwort. Die Anbindung an die echte Datenbank hat in unseren lokalen Tests funktioniert, wir haben sie allerdings nicht systematisch unter Last geprüft. Das Frontend rendert die LandingPage mit den Daten aus dem Backend.

5.4 Probleme beim Testen

Das größte wiederkehrende Problem beim Testen war der Oracle Container unter Windows. Wenn das Datenverzeichnis im Modelordner nicht da war, hat sich der Container beschwert. Wenn das Verzeichnis nach einem fehlgeschlagenen Setup nicht geleert wurde, ist der Container beim nächsten Start auch nicht wieder

hochgekommen, weil er die alten Daten erkannt hat. Als Notlösung haben wir das Datenvolume in der Composedatei zeitweise auskommentiert, damit die Datenbank jedes Mal frisch aufgebaut wird.

Das Database-Treiber Problem lag in einem unprotokollierten Konflikt zwischen dem Python-Treiber und den PL/SQL-Prozeduren der Oracle-Datenbank. Bei der Abfrage einzelner Datensätze über IDs blockierten sich der Treiber und die Datenbank-Engine bei der internen Verhandlung der Datenstrukturen komplett gegenseitig, was zu unendlichen Ladezeiten ohne Fehlermeldung führte. Um dieses Problem zu lösen, haben wir die Pakete umgangen und haben unsere Abfragen auf direktes, stabiles Raw-SQL umgestellt, wodurch die Daten nun in Millisekunden geladen werden.

6. Deployment und Installation

Eine Schritt für Schritt Anleitung für Anwender und Entwickler steht in [Anhang B](#). Hier nur die Kurzfassung.

6.1 Installation

Man braucht Docker Desktop, git und einen kostenlosen Account bei der Oracle Containerregistrierung, damit man das Datenbankimage herunterladen darf. Danach klonet man unser Repository, kopiert die Datei `.env.example` zu `.env`, trägt seine Passwörter ein und startet mit `docker compose up` den gesamten Stack. Beim ersten Mal dauert das ein paar Minuten, weil das Oracle Image groß ist und die Datenbank beim ersten Start unser Setupskript ausführt.

6.2 Konfiguration

Die wichtigste Konfigurationsdatei ist `.env` im Ordner `deployment`. Dort ist eine `.env.example` Datei vorhanden, was die benötigten Umgebungsvariablen auflistet.

6.3 Nutzung

Sobald der Stack läuft, ist das Frontend unter <http://localhost> erreichbar. Das Backend läuft intern unter `backend:8080` und wird vom Frontend automatisch angesprochen. Eine Swagger Oberfläche für die Backend API gibt es unter <http://localhost:8080>, wenn man den Port nach außen freigibt.

7. Entwicklungsprozess und Reflexion

7.1 Vorgehensweise

Wir haben uns am Anfang grob an die Phasen aus dem Exposé gehalten. Zuerst haben wir die Anforderungen aufgeschrieben und das Datenmodell entworfen. Danach haben wir das ER Diagramm gezeichnet, das SQL Setupskript geschrieben und die beiden PL SQL Pakete gebaut. Parallel dazu hat einer von uns das Frontend mit der LandingPage angefangen und einer die Flask Backend API-Endpunkte angelegt. Wir haben mit Git gearbeitet, Issues auf GitHub angelegt und die Aufgaben aufgeteilt. Datenbank, Backend und Frontend hatten jeweils einen Hauptverantwortlichen aus dem Team.

7.2 Aufgetretene Probleme

Im Verlauf des Projekts kristallisierten sich zwei Probleme heraus, die uns am meisten gebremst haben. Das waren ORDS kombiniert mit den Datenbank-Verbindungen und die Zeit. Beide beschreiben wir hier ehrlich, damit die nächste Klasse weiß, was sie erwartet.

ORDS, das Oracle Setup und die Treiber-Hürden. ORDS war von Anfang an Teil unseres geplanten Stacks, weil es unsere PL/SQL-Pakete automatisch als REST-Endpunkte verfügbar gemacht hätte. In der Theorie ist das eine sehr elegante Lösung, in der Praxis sind wir aber an mehreren Stellen hängen geblieben. Der ORDS-Container brauchte eine komplett hochgefahrte Datenbank, bevor er starten konnte, und der erste Start war jedes Mal anders. Als wir ORDS schließlich sein lassen mussten, um an anderer Stelle weiterzukommen, stießen wir beim direkten Anbinden der PL/SQL-Pakete an unser Python-Backend auf das nächste große Problem: Ein hartnäckiger Fehler im Datenbank-Treiber führte bei ID-Abrufen zu unendlichen Ladezeiten und blockierte das gesamte System. Wir mussten einen Teil der Logik auf direkte SQL-Abfragen umstellen und spezielle Cursor-Tricks anwenden, um diese Deadlocks zu umschiffen. Den ORDS-Block in unserer Composedatei haben wir absichtlich auskommentiert stehen gelassen. Wir lassen das Thema bewusst offen für die Leute, die das Projekt übernehmen. Sie können entweder einen zweiten Anlauf nehmen oder sich bewusst für unsere direkte, stabilisierte Verbindung entscheiden.

Zeit und Umfang. Schon das Exposé war ehrlich gesagt sehr ambitioniert. Zwei Frontends (Teilnehmer und Admin), Backend, PDF Dienst, KI Modul und dazu noch die Datenbank mit eigener Logik. Weil wir am Anfang sehr viel Zeit in das Setup und das Datenmodell gesteckt haben, blieb am Ende des Schuljahres deutlich weniger Zeit übrig, als wir gedacht hatten. Der PDF Dienst, die KI und das ganze Adminfrontend konnten wir nicht mehr anfangen. Im Nachhinein hätten wir den Umfang früher kleiner machen und mit dem PDF Dienst eher anfangen müssen, bevor wir das Frontend so weit gebaut haben.

Daneben gab es noch ein paar kleinere Probleme, die immer wieder aufgetaucht sind. Zum einen die schon beschriebenen Volumeprobleme beim Oracle Container unter Windows (siehe [Kapitel 5.4](#)).

7.3 Wie wir das ORDS Problem gelöst haben

Da wir Oracle REST Data Services (ORDS) in unserer Umgebung nicht zum Laufen bringen konnten, haben wir uns eine alternative und technisch anspruchsvolle Lösung überlegt, indem unser Python-Backend mithilfe des offiziellen Treibers oracledb selbstständig einen Connection-Pool aufbaut. Bei der direkten Anbindung der Datenbank stießen wir jedoch auf unerwartete Treiber-Schwierigkeiten, da der Versuch, die PL/SQL-Pakete bei ID-Abrufen direkt über die Methode `cursor.callproc()` anzusprechen, zu einer unendlichen Endlosschleife und einem vollständigen Socket-Level-Deadlock führte. Der moderne „Thin Mode“-Treiber und die Oracle-Engine blockierten sich dabei gegenseitig bei der Verhandlung der komplexen SYS_REFCURSOR-Metadatenstruktur. Um diese Hürde elegant zu nehmen, haben wir eine zweigleisige Strategie entwickelt: Wo es die Aufgabenstellung erlaubte, haben wir die fehlerhafte Metadaten-Verhandlung komplett umgangen, indem wir performantes Raw-SQL mittels `cursor.execute()` ausführten und die Daten über ein explizites Spalten-Mapping via `rowfactory` direkt in unsere Python-Klassen einspeisten. Für die verbleibenden Paket-Aufrufe implementierten wir einen gezielten Cursor-Ergänzungs-Trick, bei dem wir über `.fetchall()` den Datenstrom vollständig leersaugen, was die Oracle-Datenbank dazu zwingt, den Speicherkanal nativ zu schließen, wodurch der Deadlock ausbleibt. Durch diese Architektur erfüllt unser Stack die fachlichen Ziele des Projekts auch ohne ORDS vollends. Der große Vorteil für die Zukunft liegt darin, dass das Backend anpassungsfähig bleibt; sollte ein

nachfolgendes Team ORDS doch noch aktivieren, lassen sich die Routen mit minimalem Aufwand umstellen, da unsere Python-Datenstrukturen bereits perfekt auf das zugrundeliegende Tabellenschema abgestimmt sind.

7.4 Reflexion

Wenn wir zurückblicken, haben wir mit der Datenbank und dem Datenmodell deutlich mehr Zeit verbracht, als wir uns vorher überlegt hatten. Fachlich ist das richtig, weil das Modell jetzt solide ist und die PL SQL Pakete die Logik sauber kapseln. Das Problem dabei ist, dass uns am Ende genau die Funktionen fehlen, die man auch wirklich sieht. Also der PDF Dienst, die KI und das Adminfrontend. Geholfen hat uns auf jeden Fall, dass wir die Aufgaben klar im Team aufgeteilt haben und dass der Composestack früh stand, sodass jeder seinen Teil testen konnte. Was uns gefehlt hat, ist eine konsequente Risikobetrachtung. Wir hätten ORDS schon viel früher entweder zum Laufen bringen oder rausschmeißen müssen, und wir hätten uns früher von Funktionen verabschieden müssen, die wir uns nicht zutrauen.

8. Entscheidungen und Begründungen

8.1 Technologische Entscheidungen

Für das Frontend haben wir React 19 mit Vite, TypeScript, Tailwind und Material UI gewählt. Vite haben wir Webpack vorgezogen, weil der Entwicklungsserver viel schneller startet und die Konfiguration einfacher ist. Material UI haben wir genommen, weil einer aus dem Team schon Erfahrung damit hatte und wir so kein eigenes Designsystem bauen mussten. Im Backend haben wir uns für Flask zusammen mit flask restx entschieden, weil man damit sehr kurze und übersichtliche REST Klassen schreiben kann und automatisch eine Swagger Oberfläche bekommt. Die Datenbank Oracle war von der Schule vorgegeben.

8.2 Architekturentscheidungen

Die wichtigste Entscheidung in der Architektur war, ORDS im Laufe der Entwicklung sein zu lassen. Die ausführliche Begründung dafür steht in [Kapitel 7.2](#). Der Vorteil ist, dass unser Stack jetzt zuverlässig startet und die Geschäftslogik in der Datenbank unverändert bleibt. Der Nachteil ist, dass wir die automatische REST Generierung von ORDS verlieren und jeden Endpunkt selbst im Backend deklarieren müssen. Wir haben das Thema aber bewusst nicht endgültig zu den Akten gelegt. Die nächste Klasse kann jederzeit einen neuen Anlauf nehmen, weil die Pakete in der Datenbank schon vollständig vorbereitet sind. Eine zweite Architekturentscheidung war, das Datenmodell stark zu normalisieren, also zum Beispiel Land, Postleitzahl und Adresse als eigene Tabellen zu führen, und Geschäftsdaten über UUIDs zu adressieren. Das macht spätere Umzüge zwischen Umgebungen einfacher.

8.3 Fachliche Entscheidungen

Im Verlauf des Projekts haben wir den Funktionsumfang noch einmal eingegrenzt. Die Mussfunktionen sind weiterhin Buchung, Angebot, Rechnung und Zertifikat. Ein Kalender mit Terminen gehört ausdrücklich nicht dazu, weil das ein zusätzliches großes Modul wäre. Auf der Datenbankseite haben wir Strukturen wie Promocode und Referral bereits angelegt, damit das Datenmodell vollständig bleibt. Die passende Anwendungslogik im Backend ist aber bewusst noch nicht implementiert.

9. Übergabe an die nächste Klasse

9.1 Offene Punkte

Wenn wir das Projekt jetzt übergeben würden, sollten die folgenden Punkte in dieser Reihenfolge angegangen werden. Wir haben die Reihenfolge bewusst so gewählt, weil jeder Punkt auf dem vorherigen aufbaut.

1. Zuerst die Mockdaten im Backend vollständig durch die Persistenzschicht ersetzen. Für Kurse ist alles vorbereitet, für die User Routen müssen die Aufrufe an `user_api_pkg` noch eingebaut werden. Danach arbeitet das Backend überall mit der echten Datenbank.
2. Direkt danach in `UsersCalls` die Dummywerte gegen das Paket `user_api_pkg` austauschen, also gegen `get_all_users`, `get_user_by_id` und `create_user_record`.
3. Den PDF Dienst als eigenes Modul im Backend bauen, am besten mit `reportlab` oder `WeasyPrint`, sodass er Angebote, Rechnungen und Zertifikate erzeugen kann.
4. Eine lokale KI anbinden, zum Beispiel über `Ollama` oder `llama.cpp`, und einen Endpunkt `/ai/generate` mit fertigen Prompts für Angebote und Rechnungen bauen.
5. Das Adminfrontend bauen. Das kann entweder als eigenes Reactprojekt laufen oder als geschützte Routen unter `/admin` im bestehenden Frontend. Dabei werden Formulare für Kurs, Firma und Teilnehmer gebraucht.
6. Login und Authentifizierung umsetzen. Im Backend einen Loginendpunkt bauen, der ein JWT zurückgibt, und im Frontend die Routen schützen.
7. Über ORDS noch einmal entscheiden. Entweder den Versuch ein zweites Mal angehen und stabil aufsetzen oder die auskommentierten Blöcke aus der `Composedatei` entfernen. Wir haben das Thema bewusst offen gelassen.
8. Tests einführen. Im Backend mit `pytest`, im Frontend mit den schon eingerichteten `vitest` Werkzeugen, und idealerweise in einem GitHub Actions Workflow.

9.2 Zusätzliche Erweiterungen

Wenn die nächste Klasse mit den Pflichtthemen fertig ist und noch Zeit übrig hat, sind die Kannziele aus unserem Exposé naheliegend. Also Kursempfehlungen durch KI, Zusammenfassungen von Teilnehmerfeedback und erweiterte Statistiken im Adminbereich. Eine echte Zahlungsanbindung wäre auch denkbar, war im Exposé aber bewusst ausgegrenzt.

9.3 Tipps für die nächste Klasse

Was uns geholfen hätte, wären drei Sachen. Erstens ein einheitliches Linting und Formatting auch im Backend, damit der Code in Pullrequests immer gleich aussieht. Zweitens bessere Healthchecks in den Containern, damit man schneller sieht, wenn ein Service nicht startet. Drittens ein eigenes Makefile oder npm Skript, in dem die Setupschritte zusammengefasst sind, damit ein neues Teammitglied nicht jeden Befehl einzeln aus der Doku ablesen muss.

10. Fazit

10.1 Zusammenfassung

Am Ende des Schuljahres haben wir ein Projekt, das vor allem in der Datenbank und in der Architektur weit gekommen ist. Die Datenbank hat 16 Tabellen, fertige PL SQL Pakete und einen lauffähigen Stack mit Frontend, Backend und Oracle Datenbank in Docker. Was uns aber fehlt, sind die Funktionen, die man später wirklich sieht. Also der PDF Dienst, die KI und das Adminfrontend. Der Backendstack hat sich auch ein bisschen verändert, weil wir oracledb statt ORDS benutzen, aber das ändert nichts an den fachlichen Zielen. Die Grundlage für die Weiterarbeit steht, und genau dafür ist diese Doku gedacht.

10.2 Was wir persönlich mitnehmen

Dieses Projekt war für uns eine wertvolle Lernphase, in der wir tiefe Einblicke in den Umgang mit Docker und echten Oracle-Datenbanken gewonnen haben. Besonders lehrreich war dabei der Umgang mit den PL/SQL-Paketen, da uns fehlerhafte Treiber-Verhandlungen und blockierende Datenbank-Cursor vor massive technische Probleme gestellt haben, deren Analyse und Behebung uns im regulären Unterricht so nie begegnet wären. Gleichzeitig hat uns das Projekt gezeigt, wie entscheidend eine realistische Vorabplanung ist; wer sich zu viel vornimmt, verliert am Ende leicht das Wesentliche aus den Augen. Wir übergeben der nächsten Klasse ein absolut ehrliches Projekt mit einer technisch soliden Basis, auf der die noch fehlenden, sichtbaren Funktionen nun aufgebaut werden können. Wir wünschen unseren Nachfolgern dabei viel Erfolg.

Anhang A. Diagramme und Entwürfe

A.1 ER Diagramm

Das vollständige ER Diagramm liegt im [Repository](#) im Ordner docs und ist mit draw.io erstellt worden. Es zeigt alle 16 Tabellen und ihre 22 Beziehungen zueinander. Drei der Beziehungen sind als deferrable angelegt.

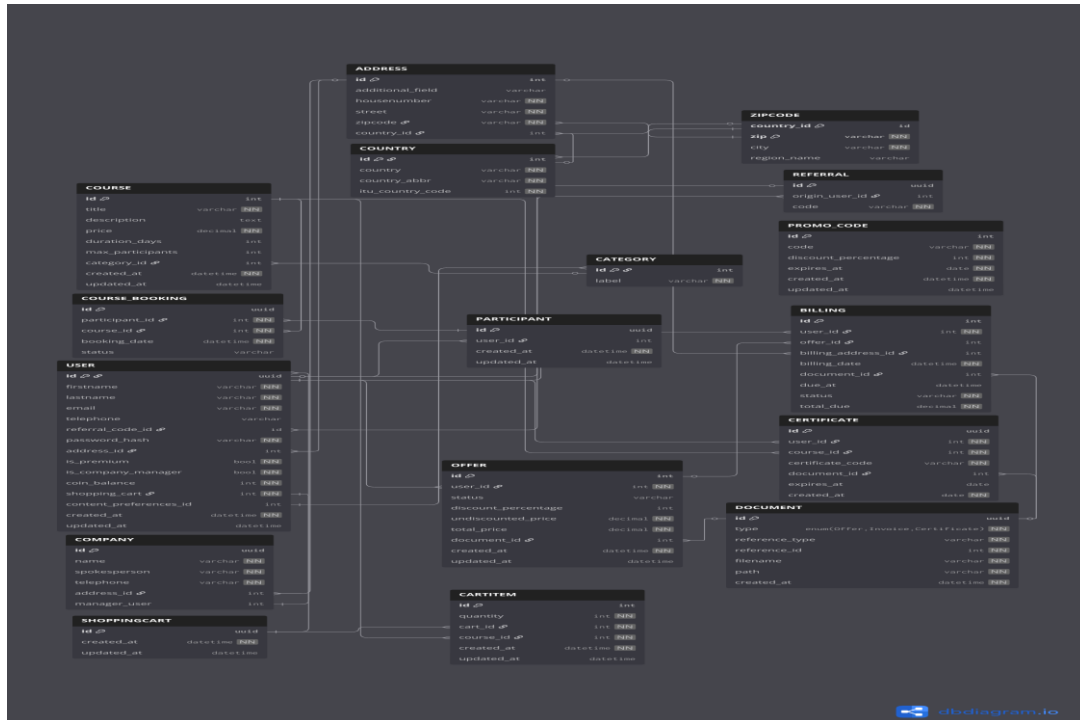


Abbildung A1. ER Diagramm Seite 1

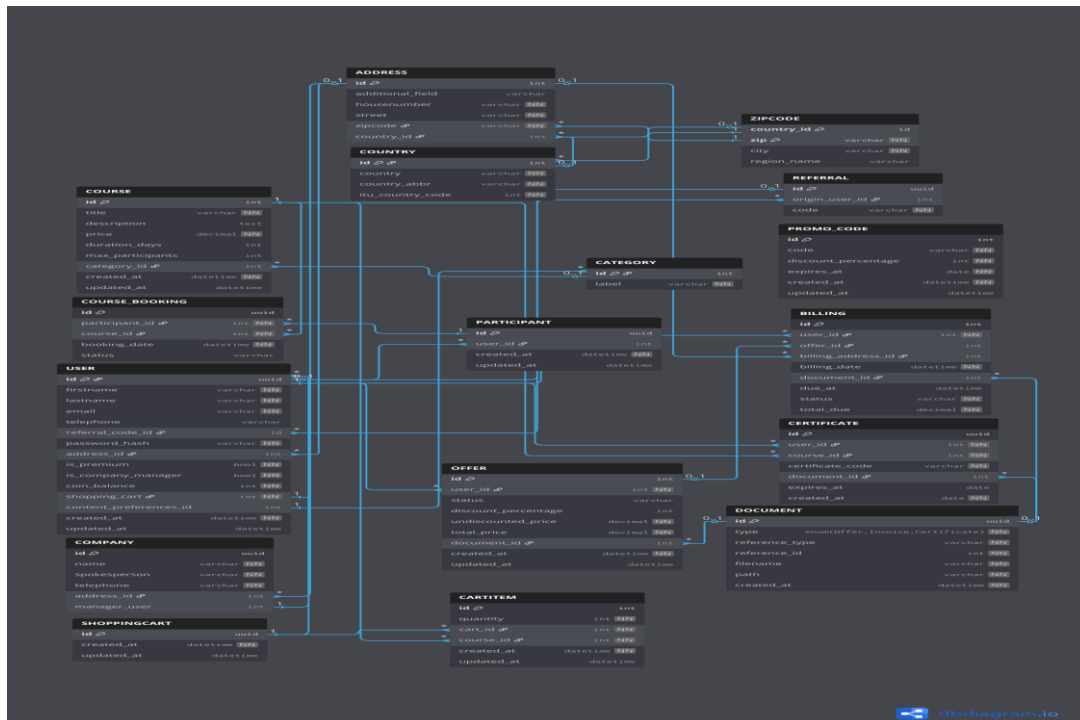


Abbildung A2. ER Diagramm Seite 2

A.2 Aktivitätsdiagramm

Das Aktivitätsdiagramm zeigt den geplanten Ablauf einer Kursbuchung. Von der Auswahl des Kurses über die Angebotserstellung und die Bezahlung bis zur Ausstellung des Zertifikats. Die Quelldatei liegt im Repository im Ordner docs.

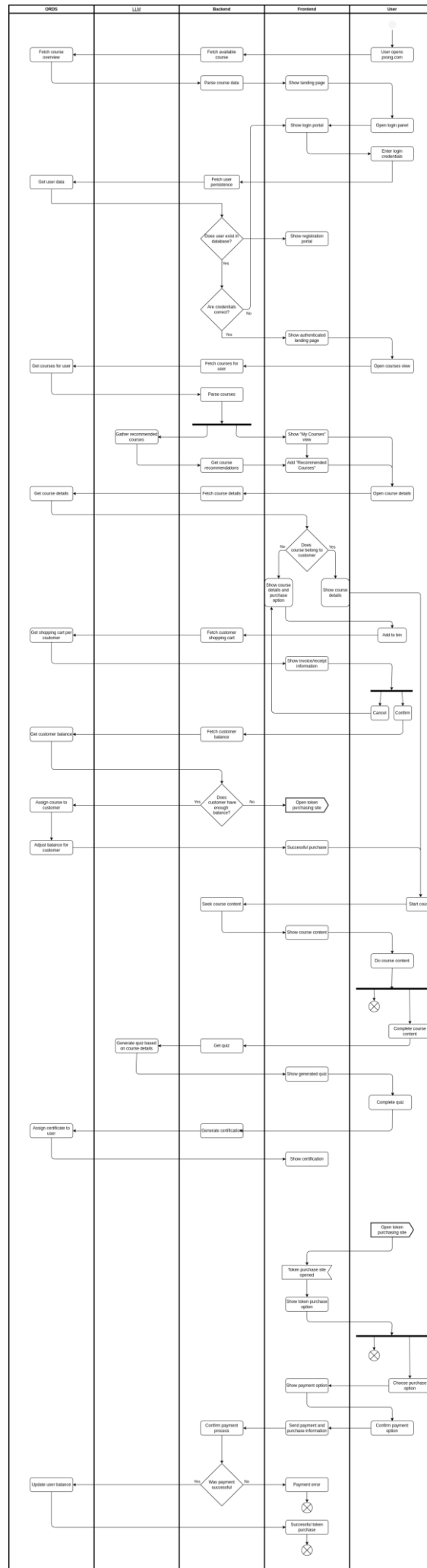


Abbildung A3. Aktivitätsdiagramm Buchungsprozess

Anhang B. Anleitung für Anwender und Entwickler

B.1 Anleitung für Anwender

Diese Anleitung beschreibt, wie ein normaler Anwender unser System benutzt. Vorausgesetzt wird, dass jemand das System schon gestartet hat, siehe dazu [Abschnitt B.2.](#)

B.1.1 Frontend öffnen

9. Den Browser starten und <http://localhost> aufrufen.
10. Auf der Startseite erscheint eine Liste der verfügbaren Kurse als waagerechtes Karussell.
11. Über die Leiste oben (TopBar) erreicht man später Login, Suche und Profil.

B.1.2 Was schon geht

- Kurse anschauen, je nach Stand mit echten Datenbankdaten oder mit Mockdaten.
- Das Aussehen zwischen hell und dunkel umschalten, je nach Einstellung des Betriebssystems.

B.1.3 Was noch nicht geht

- Sich registrieren, einloggen oder ein eigenes Profil pflegen.
- Einen Kurs wirklich buchen.
- Eigene Buchungen oder Zertifikate sehen.
- Den Adminbereich mit CRUD und Dashboard benutzen.

B.2 Anleitung für Entwickler

Diese Anleitung richtet sich an Entwickler, die das Projekt klonen, lokal starten und daran weiterarbeiten. Wir haben sie bewusst Schritt für Schritt geschrieben, damit auch jemand, der das Projekt noch nicht kennt, gut einsteigen kann.

B.2.1 Voraussetzungen

- Docker Desktop mit Linuxcontainern, mit mindestens 6 GB RAM für Docker.

- Git und ein Account bei der Oracle Containerregistrierung, damit man das Datenbankimage herunterladen darf. Der Account ist kostenlos und man muss einmal die EULA für das Image database free akzeptieren.
- Wer am Frontend arbeitet, braucht zusätzlich Node.js in Version 22, dazu pnpm oder npm.
- Wer am Backend arbeitet, braucht Python in Version 3.11 und ein virtuelles Environment. Für den oracledb Thickmode wäre auch der Oracle Instant Client nötig, aber im Standardthinmode klappt es ohne.

B.2.2 Repository klonen

```
git clone https://github.com/TimLMueller/pxsng.git
cd pxsng
git fetch --all
```

B.2.3 Datenbank starten und Stack hochfahren

12. Bei der Oracle Containerregistrierung einloggen mit dem Befehl `docker login`.
13. In den Ordner `deployment` wechseln und die Datei `.env.example` zu `.env` kopieren.
14. In der Datei `.env` mindestens `ORACLE_PWD`, `APP_USER` und `APP_USER_PASSWORD` setzen.
15. Den Stack starten mit dem Befehl `docker compose up`.
16. Beim Erststart einen Moment warten, denn die Datenbank führt jetzt unser Setupskript aus, was ein paar Minuten dauern kann.
17. Wenn etwas hängt, mit dem Befehl `docker compose logs` nachschauen, welcher Container Probleme hat.
18. Das Frontend ist danach unter <http://localhost> erreichbar.

B.2.4 Backend mit echter Datenbank verwenden

19. In `backend/.env` die drei Variablen `DATABASE_USER`, `DATABASE_PASSWORD` und `DATABASE_DSN` setzen. Bei `DATABASE_DSN` trägt man zum Beispiel `database:1521/FREEPDB1` ein.
20. Den Backendcontainer neu bauen mit `docker compose build backend` und dann starten.

21. Mit dem Befehl `curl http://localhost:8080/courses` prüfen, dass jetzt echte Daten aus dem PL SQL Paket kommen, sobald die Datenbank schon Kurse enthält.

B.2.5 Frontend lokal entwickeln

```
cd frontend
pnpm install          # oder npm install
pnpm dev              # startet Vite auf http://localhost:5173
# .env: BACKEND_URL=http://localhost:8080
```

B.2.6 Tests und Linting

```
cd frontend
pnpm lint
pnpm lint:fix
pnpm format

cd backend
python -m venv .venv && source .venv/Scripts/activate
pip install -r requirements.txt
flask --app src/main.py run --host 0.0.0.0 --port 8080
```

B.2.7 Datenbank zurücksetzen

Wenn die Datenbank Probleme macht, weil sich beim letzten Setup etwas aufgehängt hat, hilft fast immer das komplette Zurücksetzen der Volumes.

```
docker compose down -v
# zusätzlich den Inhalt von model/oracle-data/data leeren,
# falls dort noch alte Datendateien liegen.
docker compose up -d
```

B.2.8 Wo am besten anfangen

Wir empfehlen, in der gleichen Reihenfolge zu arbeiten, die in [Kapitel 9.1](#) steht. Also zuerst die Mockdaten im Backend durch die fertigen Persistenzaufrufe ersetzen, danach den PDF Dienst und die KI bauen und parallel das Adminfrontend anfangen. So baut jeder Schritt auf dem vorherigen auf.

Anhang C. Auszüge aus dem Code

C.1 Aufruf des PL SQL Pakets im Backend

```
# backend/src/Persistence/CoursesCalls.py
def GetAllCourses(self) -> list[Course]:
    conn = self.db_manager.connection
    try:
        with conn.cursor() as cursor:
            p_courses_cursor = cursor.var(oracledb.CURSOR)
            cursor.callproc(
                "course_api_pkg.get_all_courses",
                [p_courses_cursor]
            )
            result_cursor = p_courses_cursor.getvalue()
            result_cursor.rowfactory = Course
            return result_cursor.fetchall()
    finally:
        self.db_manager.release(conn)
```

C.2 Service im Frontend

```
// frontend/src/services/courseService.ts
const BACKEND_URL = process.env?.BACKEND_URL ?? 'http://backend:8080';

export const getCourses = async () => {
    const response = await fetch(`${BACKEND_URL}/courses`);
    if (!response.ok) {
        throw new Error(`Error fetching course data: ${response.statusText}`);
    }
    return response.json();
};
```

C.3 Auszug aus dem Datenbank Setup

```
-- model/oracle-data/setup/sql/setup_db_core.sql (Auszug)
PACKAGE course_api_pkg AS
    PROCEDURE get_all_courses(p_courses_cursor OUT SYS_REFCURSOR);
    PROCEDURE get_course_by_id(
        p_course_id      IN  NUMBER,
        p_course_cursor OUT SYS_REFCURSOR);
END course_api_pkg;
```